C L A U D E C O D E

A practical intro for engineers new to coding agents

Start simple.

Scale smart.

Dan Vega

Spring Developer Advocate · Broadcom

One quick disclaimer.

If you're already running five terminals with custom agent and skills...

This talk probably isn't for you. Go grab another coffee. We'll be in the fundamentals for most of this talk.

CODING AGENTS

One tool today. Most of it translates to your favorite.

Codex Gemini CLI Copilot Cursor Windsurf Grok Build Kiro

I want to describe a feeling I get these days.



Morgan Lee

@shipwright_dev · 2m

five agents in five tabs, numbered. a notification pings me when one needs an answer. I barely touch the keyboard now

12 8 46 15



Kay Greenfield

@greenfield_kay · 7m

reviewed three PRs from the gym this morning. my agents opened four more while I was in the shower

17 23 89 31



Four Terminals

@fourterminals · 11m ·

if you're not orchestrating subagents in 2026, you're already obsolete

12 31 76 18



Cronjob Carla

@cronjob_carla · 18m ·

set up a routine that triages our backlog at 6am. it closed two issues before standup and nobody noticed for a week

9 27 64 11



NullPointer Nate

@nullpointer_nate · 24m ·

20 years in this industry and I have never felt this far behind

18 42 97 23

Social media driven development.

QUICK SHOW OF HANDS…

Who's feeling overwhelmed?

It's okay. This is a safe space.

Things we're supposed to learn in coding agents.

CLAUDE.md | Hooks | Subagents | MCP | Worktrees | Plan Mode | Skills

Agents | /clear | SKILL.md | Memory | /init | Plugins | Agent Teams

Things we're supposed to learn in coding agents.

CLAUDE.md
Hooks
Subagents
MCP
Worktrees
Plan Mode
Skills
Agents
/clear
SKILL.md
Memory
/init
Plugins
Agent Teams
Harness
Checkpoints
PostToolUse
Status Line
/context
Sonnet
Permissions
Routines
Tokens
/goal
Compaction
PreToolUse
Marketplace
/resume

Things we're supposed to learn in coding agents.

CLAUDE.md
Hooks
Subagents
MCP
Worktrees
Plan Mode
Skills
Agents
/clear
SKILL.md
Memory
/init
Plugins
Agent Teams
Harness
Checkpoints
PostToolUse
Status Line
/context
Sonnet
Permissions
Routines
Tokens
/goal
Compaction
PreToolUse
Marketplace
/resume
SessionStart
Auto Mode
Context Window
Side Chat
Background Sessions
Slash Commands
Headless Mode
Opus
Dispatch
Auto-Accept Edits
Cowork
Effort Levels
Haiku
.claude/
Computer Use

the wall

...and that's just this week

You are not behind.
You don't need to learn all of this.
You need the building blocks.

Scale later. We're all on our own journey.

HELLO 👋

About me.

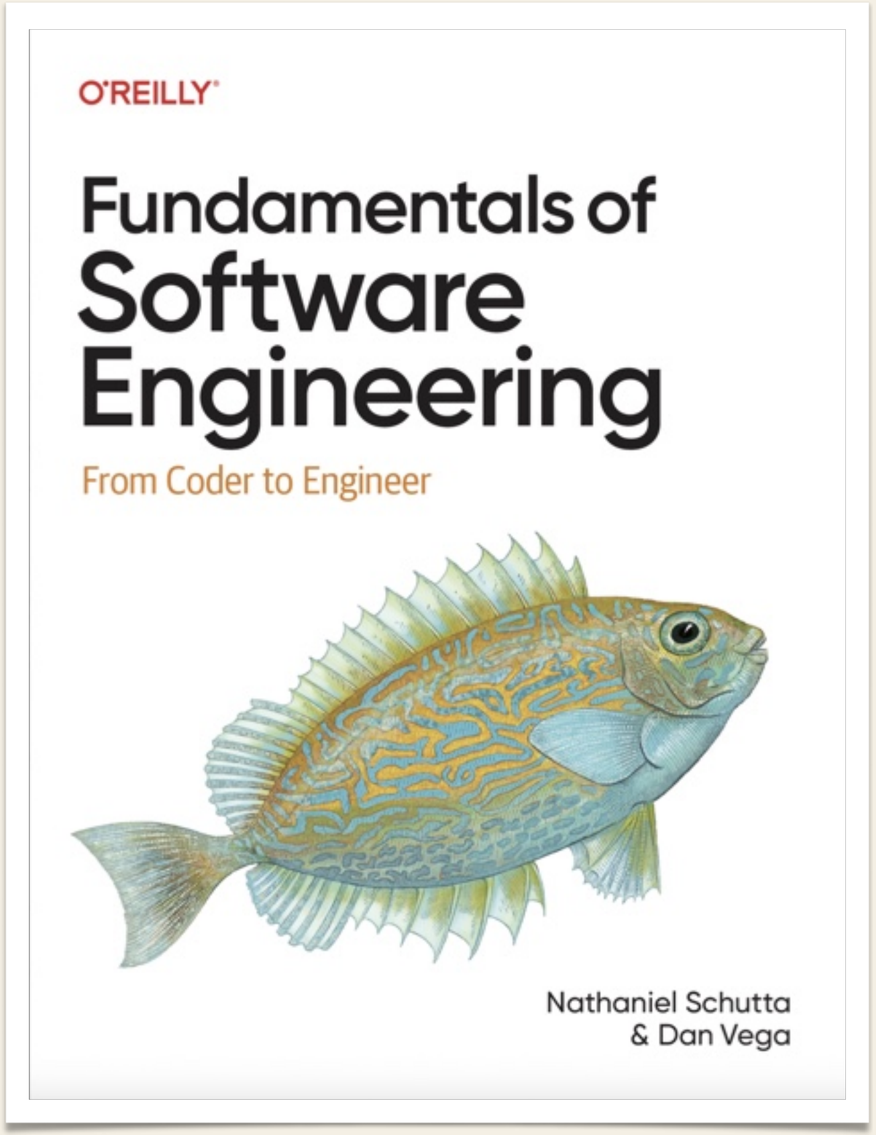
25 Years as a software engineer

Java Champion

Spring Developer Advocate · Broadcom

Author

danvega.dev



TODAY

What we'll cover.

01 **The fundamentals** — agent · harness · token · context window

02 **Getting started** — three surfaces · slash commands · permissions

03 **Context is the job** — CLAUDE.md · skills · MCP

04 **Scaling up** — subagents · hooks · goals

SECTION 01 / 04

The fundamentals.

The vocabulary everything else hangs on.

01 **The fundamentals**

02 Getting started

03 Context is the job

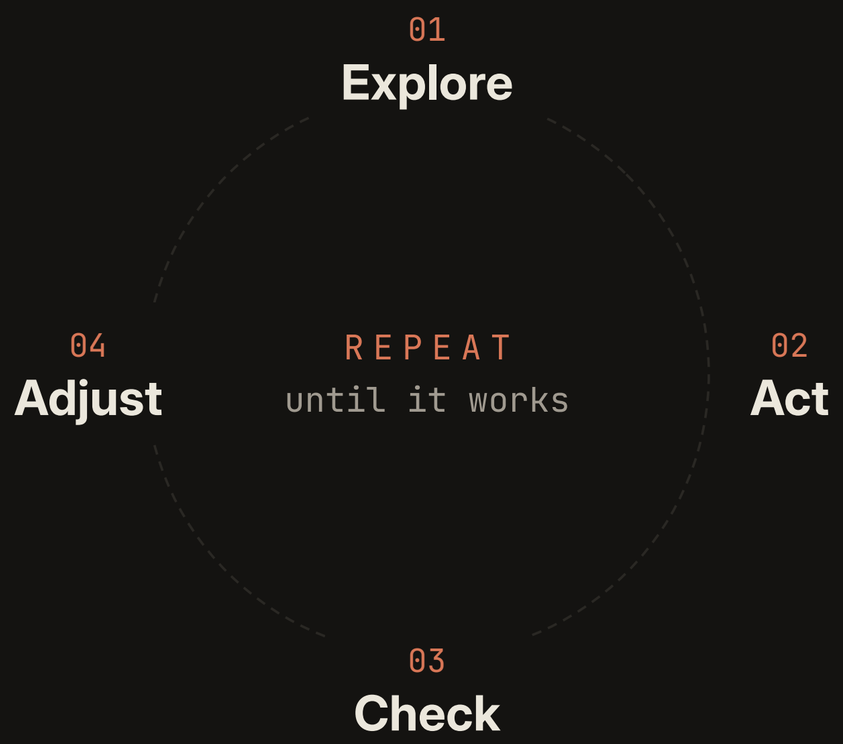
04 Scaling up

FUNDAMENTALS · 01

Agent.

AGENT

An agent runs its own loop.



`autocomplete`
suggests the next line. No loop.

`chat`
answers, then waits. You are the loop.

`agent`
runs the loop itself. You steer.

FUNDAMENTALS · 02

Harness.

HARNESS

The machinery that builds and runs the loop.

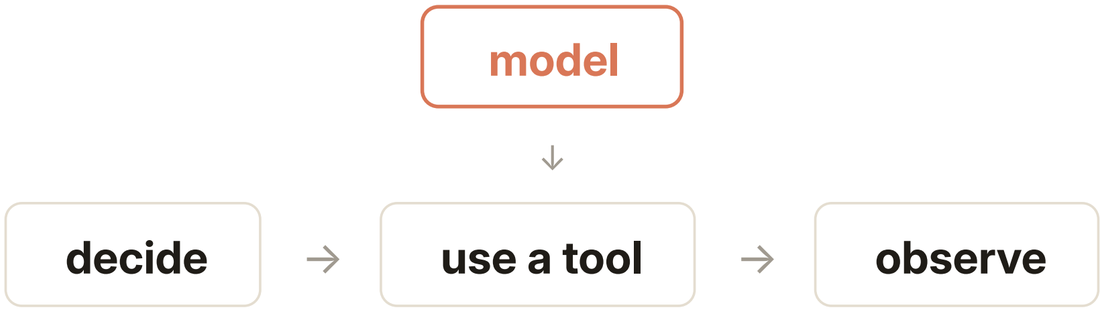
Claude Code is a harness.

So are Codex, Cursor, Gemini CLI, and Copilot's agent mode.

Same model, different harness, **different results.**

harness · **the car**

agent · **the driver**



tools
read, edit, run

context
what the model sees

safety rails
sandbox, approvals

orchestration
retries, loops

driver and car

FUNDAMENTALS · 03

Token.

TOKEN

You don't pay for words. You pay for tokens.

`Ref` `actor` `the` `payment` `service` `and` `run` `the` `tests` = 9

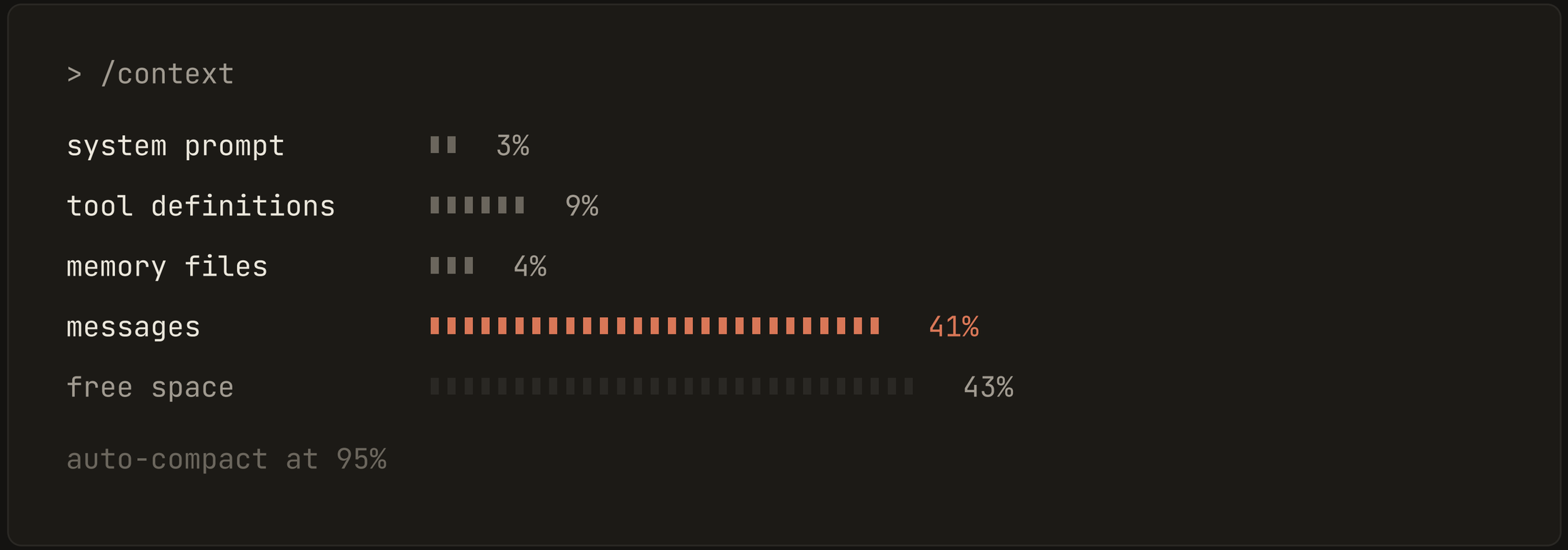
Roughly three quarters of a word. Claude reads, writes, and bills in them.

FUNDAMENTALS · 04

Context window.

CONTEXT WINDOW

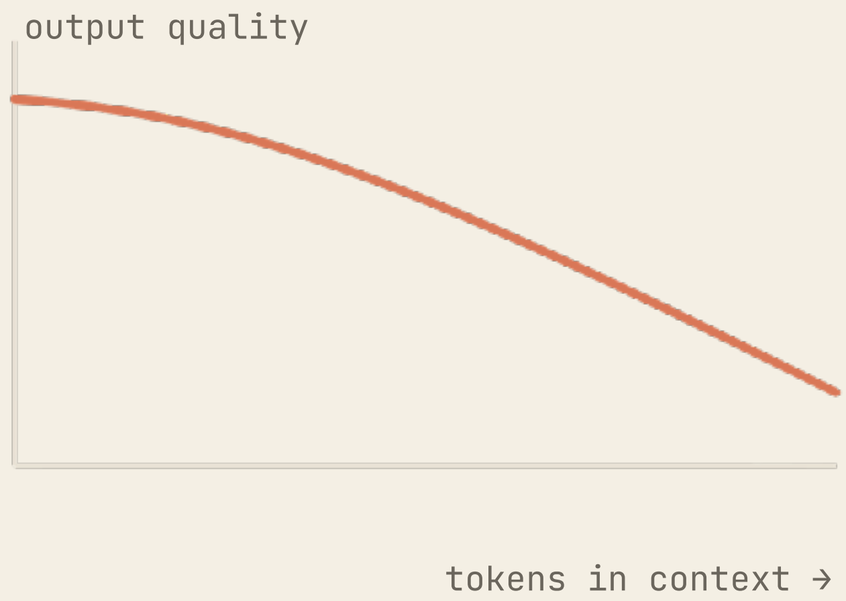
Working memory for one session.



Everything Claude is holding in its head right now. Big, but finite.

THE CATCH

Context rot.



More context is not better context.

18 frontier models tested. All 18 got worse as the window filled.

SECTION 02 / 04

Getting started.

Three surfaces, slash commands, permissions.

01 The fundamentals

02 **Getting started**

03 Context is the job

04 Scaling up

THREE WAYS IN

Same engine underneath.

`terminal`

fast, scriptable

`desktop`

parallel sessions

`web`

runs while you're away

BEFORE YOU START

Two ways to pay.

$/month

Subscription. Flat, predictable.

Pro, Max, or a Team seat. One pool across chat and Code.

$/token

API. Metered, no ceiling.

Great for scripts and CI. Pricey for daily coding.

A stray `ANTHROPIC_API_KEY` silently moves you to the right column. `/status` shows your route.

DEMO · 01

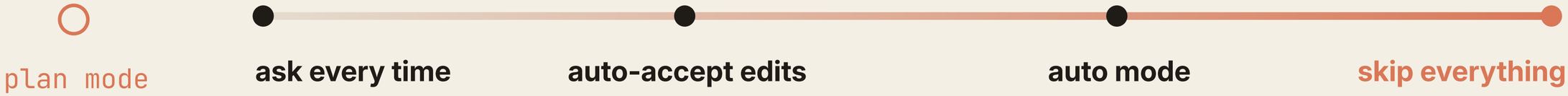
A first session.

Open a project. Run claude. Ask for one small thing. Watch the loop.

→ jump to claude code

TRUST, BUT VERIFY

Permissions.



THE ONE FOLDER TO KNOW

.claude/

Everything custom about your Claude Code setup lives here. Memory. Commands. Agents. Skills. Hooks.

```
.claude/
├── commands/your custom slash commands
├── agents/subagent configs (YAML)
├── skills/SKILL.md folders
├── hooks/lifecycle shell scripts
├── settings.jsonproject settings
└── settings.local.jsonyour overrides (gitignored)
CLAUDE.md project memory (root of repo)
```

INSIDE .CLAUDE

settings.json

One JSON file controls permissions, environment, and team-wide defaults. Checked into the repo, it travels with the project.

.claude/settings.json

```
{
"model": "sonnet",
"permissions": {
"allow": ["Bash(./mvnw *)", "Bash(git diff *)",
"Read"],
"deny": ["Bash(rm -rf *)", "Bash(mvn deploy *)",
"Read(./.env)"],
"ask": ["Bash(git commit *)", "Bash(git push *)"]
},
"env": {
"SPRING_PROFILES_ACTIVE": "test"
}
}
```

MAYBE MY FAVORITE FEATURE

Plan mode.

Hit ⇧ Tab

twice. Claude goes read-only.

The difference between "oh no" and "yes, do exactly that."

01 **Explore the codebase**

02 **Ask clarifying questions**

03 **Write a markdown plan**

04 **You approve · it executes**

DEMO · 02

Permissions and plan mode.

A new project, from first prompt to trusted setup.

→ jump to claude code

SECTION 03 / 04

Context is the job.

Steering Claude with what you already know.

01 The fundamentals

02 Getting started

03 **Context is the job**

04 Scaling up

This is where you can take a coding agent from a nice parlor trick to an irreplaceable teammate.

The model is only as good as the context you give it.

THREE GAPS, THREE FIXES

Three ways the model comes up short.

GAP 01

CLAUDE.md

What it should always know about this project.

GAP 02

Skills

What it should know how to do.

GAP 03

MCP

What it cannot see from where it sits.

PROJECT MEMORY

CLAUDE.md

A plain markdown file at the root of your repo. Claude reads it every session, automatically.

What this project is

The stack and key conventions

How to run, test, and build

Gotchas: what you wish someone told you on day one

20 minutes writing a good one = every future session is sharper.

CLAUDE.md

```
# spring-petclinic
Java 21 · Spring Boot 3.4 · PostgreSQL · Maven
## Conventions
- Constructor injection only
- Records for DTOs
- Tests use Testcontainers, not H2
## Run it
$ ./mvnw spring-boot:run
## Gotchas
- Migrations live in db/migration, not resources/
```

Write one that actually helps.

VAGUE · DON'T

"Write good code."

"Follow best practices."

"Format things properly."

"Test your changes."

Claude already tries to do all of this. Zero signal.

SPECIFIC · DO

"Use 2-space indent, no semicolons."

"Constructor injection only."

"DTOs are records in model/."

"Run ./mvnw verify before done."

Concrete rules Claude can't guess. Real signal.

Structure it. Headings and bullets, not prose.

Keep it tight. Every line costs context.

Revisit it. Fix the file, not just the chat.

KEEP IT LEAN

Progressive disclosure.

Point at the details instead of pasting them. Claude reads a file only when the task needs it.

CLAUDE.md

```
# Payments API
Constructor injection only.
Run ./mvnw verify before done.
Testing conventions: @docs/testing.md
DB migrations: @docs/migrations.md
Release process: @docs/release.md
```

Loaded up front: 8 lines. Loaded on demand: everything else.

GAP 02 · WHAT IT SHOULD KNOW HOW TO DO

A skill is a folder with a `SKILL.md`.

Expertise packaged once, pulled in on demand. Not crammed into every prompt.

SKILLS

What one looks like.

The description is the trigger. Claude reads it to decide when the skill applies.

The body is the playbook. Steps, conventions, commands.

It's just markdown. In the repo, reviewed and versioned like code.

.claude/skills/readme/SKILL.md

```
---
name: readme
description: Write, rewrite, or review
a README so it serves its actual
reader. Use whenever the task is
creating or improving a README.
---
A README has one job: get the reader
to what they came for in under a minute.
## Ground rules
- Open with one plain sentence saying
what the project is and who it's for
- Verify every command against the repo
- No hype. Show a code sample instead.
```

skill.md

SKILLS

If you explain it twice, make it a skill.

`new-endpoint`	scaffold a REST endpoint the way this team does it
`db-migration`	write a Flyway migration, naming and all
`code-review`	review a diff against the team's checklist
`release-notes`	turn merged PRs into a changelog, in house style

SKILLS IN THE WILD

One video, nine skills.

PLAN

video-develop-idea

Is this worth making? Research demand, find the angle, scope it.

video-packaging

Lock the title, thumbnail concept, and first 30 seconds.

video-project

Scaffold the project folder from my template.

→

BUILD

video-demo-design

Design and build the demo code the camera sees.

video-rough-cut

Trim dead air, filler words, and retakes.

video-motion-graphics

Add lower thirds, titles, callouts, punch-in zooms.

→

PUBLISH

video-thumbnail

Render the final thumbnail in the channel's style.

video-shorts

Chop it into vertical shorts with burned-in captions.

video-x-clips

Cut clips for X with the post copy written.

Also in the library: Spring Boot 4 reference skills (modular-auto-config, rest-test-client, jackson-3), writing skills (readme, newsletter-publish, blog-seo-optimize), and the Spring Office Hours episode pipeline.

GAP 03 · WHAT IT CANNOT SEE FROM WHERE IT SITS

MCP is a standard plug for outside tools.

One protocol. Any tool that speaks it, Claude can see and use.

MCP

What connecting looks like.

One command to connect. The server does the rest.

Claude discovers the tools. The server advertises what it can do.

You stay in charge. Tool use runs through the same permissions.

terminal

```
$ claude mcp add danvega \
https://mcp.danvega.dev
✓ Connected to danvega
✓ 4 tools discovered:
get_posts · get_talks ·
get_courses · search_content
```

MCP

Wherever Claude is guessing, plug in the source.

`github`
read the real issue, comment on the real PR

`postgres`
query the actual schema, not a guess at it

`chrome-devtools`
drive a real browser: live DOM, console, network

`figma`
build from the design file, not a screenshot

`cloudflare`
manage workers, DNS, and logs on your real account

`context7`
current docs for your framework version, not training data

CONTEXT IS THE JOB

Skills versus MCP.

SKILLS

Teach Claude how.

Markdown playbooks for repeatable work. Static. Can't reach outside the repo.

```
"how we write Flyway migrations"
```

MCP

Give Claude tools.

Live connections to tickets, databases, APIs. Capabilities. They don't teach taste.

```
"read the production schema"
```

TOGETHER MCP reads the real schema. The skill writes the migration your way.

DEMO · 03

Skills and MCP.

Teach it a skill, then plug in `mcp.danvega.dev`.

→ jump to claude code

SECTION 04 / 04

Scaling up.

One session to many. Watching to delegating.

01 The fundamentals

02 Getting started

03 Context is the job

04 **Scaling up**

SCALING UP

An agent is a file.

.claude/agents/code-reviewer.md

```
---
name: code-reviewer
description: Reviews code for quality
tools: Read, Glob, Grep
model: haiku
---
You are a code reviewer. Give specific,
actionable feedback.
```

Its own prompt
you decide how it behaves

Only these tools
read only, in this case

A cheaper model
haiku for mechanical work

Its own context window
the whole reason to use one

It reads 40 files. You get back a paragraph.

DELEGATING SAFELY

Hooks. Hard rules, not vibes.

CLAUDE.md

Advises.

The model reads it. The model can drift.

HOOKS

Enforce.

A script at a lifecycle point. Runs every time, no judgment call.

The more you delegate, the more you want rules that cannot be talked out of.

FROM WATCHING TO DELEGATING

/goal

Set the outcome, not the task. Claude keeps working, turn after turn, until it gets there.

KNOW THE DIFFERENCE

/goal is not spec driven development.

SPEC DRIVEN

The decisions, written down first.

A full spec up front. The agent builds to it. The spec is the contract.

/goal

A termination condition.

Keep going until this is true. Not a planning tool.

Don't reach for it when the task needs design decisions you haven't made yet.

WORK THAT REPEATS

Loops and routines.

Recurring and scheduled work. Your laptop does not need to be on.

schedule	api	webhook
cron-like	fire from CI	GitHub events

DEMO · 04

Agent view and /goal.

Watch Claude keep working after I stop typing.

→ jump to claude code

Remember the wall? You know the ones that matter.

CLAUDE.md
Hooks
Subagents
MCP
Worktrees
Plan Mode
Skills
Agents
/clear
SKILL.md
Memory
/init
Plugins
Agent Teams
Harness
Checkpoints
PostToolUse
Status Line
/context
Sonnet
Permissions
Routines
Tokens
/goal
Compaction
PreToolUse
Marketplace
/resume
SessionStart
Auto Mode
Context Window
Side Chat
Background Sessions
Slash Commands
Headless Mode
Opus
Dispatch
Auto-Accept Edits
Cowork
Effort Levels
Haiku
.claude/
Computer Use

the wall

You are not behind. Start simple, scale later. **We're all on our own journey.**

SAVE YOURSELF A FEW WEEKS

Mistakes I made so you don't have to.

01 **Vague one-line prompts.** "Fix the bug." Cool. Which one? Where? What did you try? Be specific.

02 **Skipping plan mode for real work.** Five minutes of planning saves an hour of "what did you just do?"

03 **Never running /clear.** Stale context bleeds into new tasks. Wipe between unrelated jobs.

04 **No CLAUDE.md.** Re-explaining your stack every session. Twenty minutes, write it once.

05 **Trusting the first answer.** Read the diff. Run the tests. Push back. Claude likes the pushback.

06 **Reaching for Skills/MCP/Agents too early.** First learn what Claude does well bare. Then add scaffolding.

gitHub.com/danvega/claude-code-start-simple

The code from this talk

That's it.

Thanks. Questions?

Dan Vega

Spring Developer Advocate · Broadcom

danvega.dev · @therealdanvega